Interrupt1

HW1. Following events will cause interrupts in the system. What interrupt number will be assigned to each event? For system call interrupt, also give the system call number.

HW2. Change "drivers/input/keyboard/atkbd.c" as follows. What happens and why does this happen? Show the sequence of events that happen when you hit a key in a normal Linux kernel

```
Static irqreturn_t atkbd_interrupt(…){
    return IRQ_HANDLED; // Add this at the first line
    ….
}
```

Result

아래의 사진과 같이 atkbd_interrupt함수의 첫번째 줄에서 바로 return을 하도록 해 주었다.

```
352 static irqreturn_t atkbd_interrupt(struct serio *serio, unsigned char data,
353                         unsigned int flags)
354 {
355         return IRQ_HANDLED;
356         struct atkbd *atkbd = serio_get_drvdata(serio);
357         struct input_dev *dev = atkbd->dev;
```

그리고 이 내용이 Kernel에 실제로 반영될 수 있도록 하기 위해서 다음과 같이 변경 내용을 컴파일 해 준 뒤, 생성된 파일을 `myLinux`에 실제 적용될 수 있도록 이동해 주었다.

```
$ make bzImage
$ cp arch/x86/boot/bzImage /boot/bzImage
$ reboot
```

그 결과 로그인 창에서부터 키보드 입력이 되지 않는 것을 확인할 수 있었다.

원인

`drivers/input/keyboard/atkbd.c/atkbd_interrupt()`는 Keyboard Interrupt의 ISR2(Interrupt Service Routine)의 동작코드가 존재하는 곳이다.

기존 코드에서는 키보드 입력에 대한 Interrupt 처리를 한 후에 `IRQ_HANDLED`를 반환하지만, 우리가 고친 코드에서는 키보드 입력에 대한 Interrupt처리없이 완료가 되므로 아무런 문자가 입력되지 않는 것이다.

HW3. Change the kernel such that it prints “x pressed” for each key pressing, where x is the scan code of the key. After you change the kernel and reboot it, do followings to see the effect of your changing.

> **Code**
>
> 우선 Keyboard Interrupt를 처리하는 `drivers/input/keyboard/atkbd.c/atkbd_interrupt()`부분을 다음과 같이 고쳐 주었다.

```
static irqreturn_t atkbd_interrupt(struct serio *serio, unsigned char data,
                        unsigned int flags)
{
        struct atkbd *atkbd = serio_get_drvdata(serio);
        struct input_dev *dev = atkbd->dev;
        unsigned int code = data;
        printk("%x pressed!! \n", code);
        int scroll = 0, hscroll = 0, click = -1;
        int value;
        unsigned char keycode;

#ifdef ATKBD_DEBUG
        printk(KERN_DEBUG "atkbd.c: Received %02x flags %02x\n", data, flags);
#endif
```

> 즉, `printk("%x pressed\n", code);`부분을 통해 내가 Keyboard로 입력한 data를 출력 할 수 있도록 바꾸어 주고 마찬가지로 컴파일하여 Kernel에 적용될 수 있도록 해 주었다.

```
# make bzImage
# cp arch/x86/boot/bzImage /boot/bzImage
# reboot
```

> **Setting**
>
> 이 경우 해당 내용이 바로 적용되지는 않았는데, 이 이유는 현재 콘솔의 log level이 printk함수의 log level보다 작기 때문에 출력되는 문자들이 콘솔 화면에 나타나지 않게 된다.
>
> default log level은 다음 명령어를 통해 확인할 수 있다.

```
localhost linux-2.6.25.10 # cat /proc/sys/kernel/printk
1        4        1        7
```

> 그 결과 출력은 위와 같이 1, 4, 1, 7로 나타나는 것을 확인할 수 있고 이 결과는 순서대로 다음을 의미한다.
>
> - `Console log level`: 이 값보다 숫자가 작은 메시지들을 Console에 출력
> - `Default Message log level`: 우선순위가 없는 message들의 Log Level, printk함수를 사용하면서 별도의 Log Level을 사용하지 않을 경우 이 값으로 설정된다.
> - `Min Console log level`: Console Log Level이 설정될 수 있는 최솟값
> - `Default Console log level`: Console Log Level용 기본 값
>
> 즉, 여기서 Default Message log level이 Console log level보다 컸기 때문에 printk의 내용이 콘솔에 출력되지 않았던 것이므로 Console log level을 8로 설정해 주었다.

```
localhost linux-2.6.25.10 # echo 8 > /proc/sys/kernel/printk
localhost linux-2.6.25.10 # cat /proc/sys/kernel/printk
8       4       1       7
```

Result

```
localhost ~ # echo 8 > /proc/sys/kernel/printk
localhost ~ # 9c pressed!!
1c pressed!!

localhost ~ # 9c pressed!!
20 pressed!!
da0 pressed!!
1e pressed!!
a9e pressed!!
21 pressed!!
fa1 pressed!!
```

HW4. Change the kernel such that it displays the next character in the keyboard scancode table. For example, when you type "root", the monitor would display "tppy". How can you log in as root with this kernel?

Code

마찬가지로 Keyboard Interrupt를 처리하는 `drivers/input/keyboard/atkbd.c/atkbd_interrupt()`부분을 다음과 같이 고쳐 준다.

```
static irqreturn_t atkbd_interrupt(struct serio *serio, unsigned char data,
                        unsigned int flags)
{
        struct atkbd *atkbd = serio_get_drvdata(serio);
        struct input_dev *dev = atkbd->dev;
        unsigned int code = data + 1;
        int scroll = 0, hscroll = 0, click = -1;
        int value;
        unsigned char keycode;
```

그 후 마찬가지로 변경 내용을 Kernel에 적용해 준다

```
# make bzImage
# cp arch/x86/boot/bzImage /boot/bzImage
# reboot
```

Result

```
This is localhost.unknown_domain (Linux i686 2.6.25.10) 22:04:59

localhost login: tppya _
```

위 그림과 같이 root를 입력하면 tppy가 입력되는 것을 확인할 수 있다.

HW5. Define a function "mydelay" in init/main.c which whenever called will stop the booting process until you hit 's'. Call this function after do_basic_setup() function call in kernel_init() in order to make the kernel stop and wait for 's' during the booting process. You need to modify atkbd.c such that it changes exit_mydelay to 1 when the user presses 's'.

Code

```
int exit_mydelay;
void mydelay(char *str){
        printk(str);
        printk("enter s to continue\n");
        exit_mydelay = 0;
        for(;;) {       // wait here until the user press 's'
                msleep(1); // sleep 1 micro second
                if(exit_mydelay == 1) // if the user press 's' break_
                        break;
        }
}
```

우선 `init/main.c`에 위와 같은 함수, global변수를 선언해 준다.
이 함수는 keyboard에 `s`가 눌릴 때까지 sleep하도록 하는 함수이다.

```
static int __init kernel_init(void * unused)
{
        lock_kernel();
```

...

```
        do_basic_setup();
        mydelay("after do basic setup in kernel_init\n"); // wait here
```

그리고 `init/main.c/kernel_init()`함수의 `do_basic_setup();`코드 밑에 위에서 선언해준 mydelay 함수를 실행하도록 해 준다.

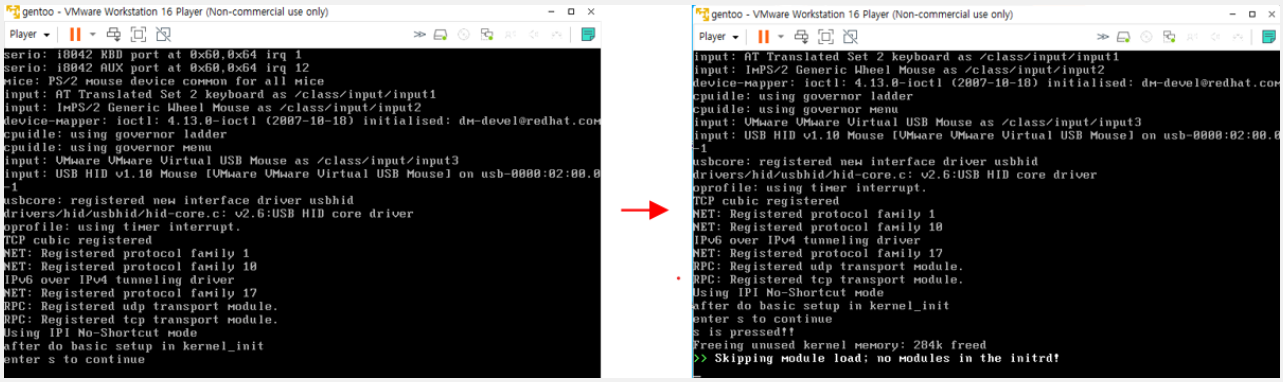
```
extern int exit_mydelay; // declare as extern since it is defined in main.c
static irqreturn_t atkbd_interrupt(struct serio *serio, unsigned char data,
                        unsigned int flags)
{
        struct atkbd *atkbd = serio_get_drvdata(serio);
        struct input_dev *dev = atkbd->dev;
        unsigned int code = data;

        if (code == 31){
                printk("s is pressed!! \n");
                exit_mydelay = 1;
        }
```

이 후, Keyboard Interrupt를 처리하는 `drivers/input/keyboard/atkbd.c/atkbd_interrupt()`부분을 위와 같이 고쳐 주어 `s`에 해당하는 31번 Keyboard가 입력되면 `exit_mydelay`가 1이 되도록 고쳐 주었다.

마지막으로 변경 내용을 Kernel에 적용해 준다

```
# make bzImage
# cp arch/x86/boot/bzImage /boot/bzImage
# reboot
```

Result



그 결과 위와 같이 keyboard에 `s`를 누르기 전에는 booting이 되지 않고 대기하고 있는 것을 확인할 수 있었다.

HW5-1) Add mydelay before do_basic_setup(). What happens and why?

Code

```
822 static int __init kernel_init(void * unused)
823 {
824         lock_kernel();
```

...

```
850         mydelay("before do basic setup in kernel_init\n"); // wait here
851         do_basic_setup();
852         mydelay("after do basic setup in kernel_init\n"); // wait here
```

이번에는 위와 같이 `do_basic_setup();`함수 위에 `mydelay()`함수를 실행하도록 추가적으로 설정해 보았다.

Result

gentoo - VMware Workstation 16 Player (Non-commercial use only)

Player

```
SLUB: Genslabs=12, HWalign=64, Order=0-1, MinObjects=4, CPUs=1, Nodes=1
Calibrating delay using timer specific routine.. 6193.99 BogoMIPS (lpj=12387999)
Mount-cache hash table entries: 512
CPU: L1 I cache: 32K, L1 D cache: 32K
CPU: L2 cache: 256K
CPU: L3 cache: 12288K
Intel machine check architecture supported.
Intel machine check reporting enabled on CPU#0.
Compat vDSO mapped to ffffe000.
Checking 'hlt' instruction... OK.
SMP alternatives: switching to UP code
Freeing SMP alternatives: 17k freed
ACPI: Core revision 20070126
Parsing all Control Methods:
Table [DSDT](id 0001) - 3598 Objects with 401 Devices 1443 Methods 277 Regions
 tbxface-0598 [00] tb_load_namespace     : ACPI Tables successfully acquired
evxfevnt-0091 [00] enable                : Transition to ACPI mode successful
CPU0: Intel(R) Core(TM) i5-10500 CPU @ 3.10GHz stepping 03
Total of 1 processors activated (6193.99 BogoMIPS).
ENABLING IO-APIC IRQs
..TIMER: vector=0x31 apic1=0 pin1=2 apic2=-1 pin2=-1
Brought up 1 CPUs
before do basic setup in kernel_init
enter s to continue
```

그 결과 Keyboard에 s를 입력해도 전혀 booting이 되지 않는 것을 확인할 수 있었다.

원인

basic_setup()전에는 Keyboard Interrupt를 받을 수 있는 기본적인 Setting도 안되어 있을 것이라고 추측된다.

HW6. Which function call in atkbd_interrupt() actually displays the pressed key in the monitor?

Code

```
353 static irqreturn_t atkbd_interrupt(struct serio *serio, unsigned char data,
354                                    unsigned int flags)
355 {
356         struct atkbd *atkbd = serio_get_drvdata(serio);
357         struct input_dev *dev = atkbd->dev;
358         unsigned int code = data;
```

...

```
496                                 input_event(dev, EV_KEY, keycode, value);
497                                 input_sync(dev);
```

아마 `atkbd_interrupt()`함수에서 input_sync(dev);에서 실제 monitor에 출력하도록 하는 것이 아닐까 싶다.

HW6-1. What are the interrupt numbers for divide-by-zero exception, keyboard interrupt, and "read" system call? Where is ISR1 and ISR2 for each of them (write the exact code location)? Show their code, too.

> 우선 각 Interrupt에 대한 구현 위치는 ISR1과 ISR2에서 다음 다음과 같다.
>
> | - | Interrupt Number | ISR1 | ISR2 |
> |---|---|---|---|
> | divide-by-zero exception | 0 | divide_error | do_divide_error |
> | keyboard interrupt | 33 | interrupt[1] | atkbd_interrupt |
> | read system call | 128 | system_call | sys_read |
>
> 또한 이때, ISR1과 ISR2의 구현 위치는 다음과 같다.
>
> | - | 위치 |
> |---|---|
> | ISR1 | `arch/x86/kernel/entry_32.S` |
> | ISR2-do_divide_error | `arch/x86/kernel/traps.32.c` |
> | ISR2-keyboard interrupt | `drivers/input/keyboard/atkbd.c` |
> | ISR2-read system call | `fs/read_write.c` |
> | ISR2-... | `...` |
>
> 즉, ISR1은 `arch/x86/kernel/entry_32.S`에 존재하지만, ISR2는 다양한 place에 존재한다.